

Contents

1	Introduction	2
2	Problem Statement	2
3	Problem Review	2
4	Methodology	3
5	Design	5
6	Tool Used	6
7	Team Member Contributions	6
8	Timeline	6

1 Introduction

This project aims to provide an interactive and visual understanding of the convolution operation, a fundamental concept in signal processing. Convolution is a mathematical operation used to analyze the output of a Linear Time-Invariant (LTI) system when both the input signal and the system's impulse response are known. Understanding convolution is essential for tasks ranging from basic signal manipulation to complex system design in fields such as communications, audio processing, and control systems.

In continuous-time systems, convolution involves two signals: the input signal $x(t)$ and the impulse response $h(t)$. The convolution integral for continuous-time signals is expressed as:

$$y(t) = (x * h)(t) = \int_{-\infty}^{\infty} x(\tau)h(t - \tau) d\tau$$

The above equation computes the output signal $y(t)$ by integrating the product of the input signal and the time-shifted impulse response over all time values.

For discrete-time systems, where signals are represented by sequences, the convolution sum is used to calculate the output. The discrete-time convolution is expressed as:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k]$$

This summation computes the output signal $y[n]$ at each discrete time index n by summing the product of the input signal $x[k]$ and the shifted impulse response $h[n - k]$ values. Thus, this convolution project provides an interactive GUI to visualize and experiment with convolution in both discrete-time and continuous-time systems, bridging theory with practical application.

2 Problem Statement

This project focuses on creating a graphical user interface (GUI) to visually demonstrate the convolution operation between an input signal and an impulse response, supporting both discrete-time and continuous-time signal modes.

For discrete-time, users will enter signal vectors along with their starting indices. In continuous-time, users will select from various standard signal types (such as impulse, step, triangular, rectangular, sawtooth) and define key parameters like start time, end time, and amplitude. The application will compute and animate the convolution process, displaying three key plots: the input signal, the mirrored and shifting impulse response, and the output signal evolving in real time.

The project will include robust input validation, user-friendly controls (including adjustable animation speed and a reset function), and clear visual feedback. The solution will be implemented using MATLAB or Python, with an appropriate GUI framework.

For bonus marks, additional features may be incorporated:

- **Animation Controls:** A slider to adjust the speed of the animation, with a reset function to clear all plots and fields.
- **Compute Correlation:** A button to compute and display the correlation between the two input signals.

3 Problem Review

Convolution is a cornerstone of signal processing, especially for analyzing Linear Time-Invariant (LTI) systems. Solving related problems demands a solid grasp of the following areas:

1. Convolution Theory:

A clear understanding of convolution in both discrete-time and continuous-time systems, including the convolution sum and integral, is required for implementing the algorithm correctly.

2. **Signal Processing:**

Knowledge of various signal types (impulse, step, triangular, rectangular, sawtooth) and the ability to specify their parameters (such as time and amplitude) is necessary for accurate user input.

3. **Programming Skills (MATLAB & Python):**

Proficiency in MATLAB or Python is essential to develop the GUI and implement the convolution algorithm. Familiarity with GUI frameworks like MATLAB App Designer or Python's Tkinter/PyQt and plotting tools like MATLAB tools or Matplotlib is required. The solution has been implemented using MATLAB.

4. **Mathematics for Signal Processing:**

A strong grasp of mathematical operations such as integration (for continuous-time) and summation (for discrete-time) is needed to perform the convolution calculations accurately.

5. **User Interface Design:**

An understanding of user interface (UI) design principles, including input validation, error handling, and providing real-time feedback, is crucial for building an intuitive and smooth user experience.

6. **Animation and Visualization:**

Knowledge of real-time animation techniques and how to visualize the evolving convolution process will help users understand how the input signal, impulse response, and output are related.

7. **Algorithm Development:**

A clear understanding of how to develop and optimize the convolution algorithm, while managing edge cases and ensuring accuracy, is central to the project's success.

8. **Bonus Features:**

For bonus marks, knowledge of adding additional features, such as adjusting animation speed and computing the correlation between the input signals, will be beneficial.

Here is how you can write the **Methodology**, **Design**, and **Tools Used** sections based on your ConvolutionVisualizer MATLAB App.

4 Methodology

The objective of this project is to visually demonstrate the concepts of convolution and cross-correlation for both discrete-time and continuous-time signals using a graphical interface. The process involves the following steps:

1. **Input Collection:**

- For discrete-time, vectors and their starting indices are collected as user input.
- For continuous-time, signal types (Impulse, Step, Rectangular, etc.), start/end times, and amplitudes are collected.

Discrete-Time	Continuous-Time
Input Vector:	[1 0 2 3 1 1 1]
Start Index:	0
Impulse Vector:	[1 2 7 4 1 1]
Start Index:	0

Figure 1: For discrete time

Discrete-Time	Continuous-Time
Signal Type:	Rectangular Pulse ▼
Start Time:	Impulse
End Time:	Step
Amplitude:	Triangular Pulse
Signal Type:	Rectangular Pulse
Signal Type:	Sawtooth Pulse
Start Time:	0
End Time:	1
Amplitude:	1

Figure 2: For continuous time

2. Signal Generation:

- Signals are generated based on user-defined parameters using custom mathematical functions.

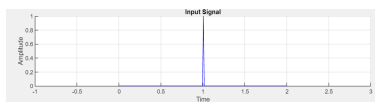


Figure 3: impulse

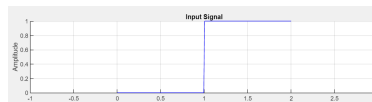


Figure 4: step

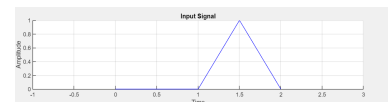


Figure 5: triangular

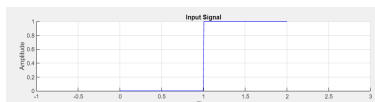


Figure 6: rectangular

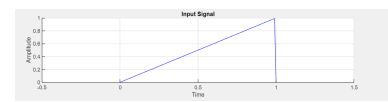
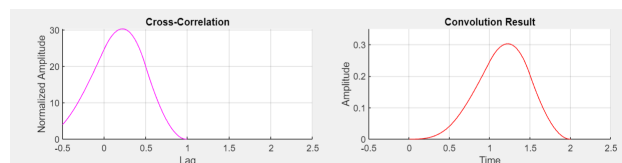


Figure 7: sawtooth

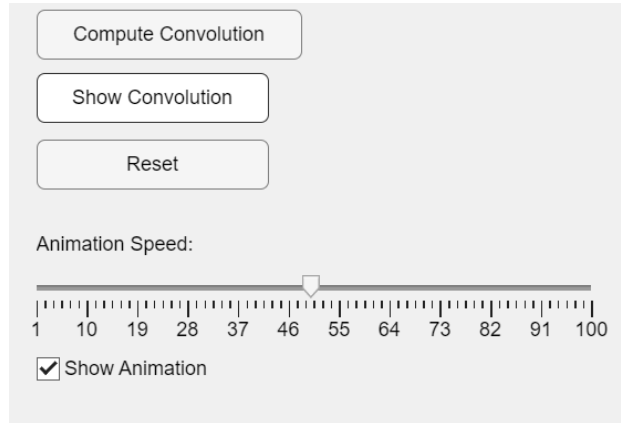
3. Computation:

- Convolution is calculated using MATLAB's `conv()` function.
- Cross-correlation is computed using MATLAB's `xcorr()` function.



4. Visualization:

- All signals and results are plotted using multiple `UIAxes` components.
- Animation of the convolution process is shown step-by-step to enhance understanding.
- A toggle button switches between convolution and correlation views for better comparison.



5 Design

The application follows a tabbed GUI layout consisting of two primary tabs:

- **Discrete-Time Tab:** Accepts vector inputs and starting indices for both input and impulse signals.
- **Continuous-Time Tab:** Provides dropdowns and numeric inputs to select and parameterize standard signal types.

UI Components include:

- **Input Fields:** For entering vectors, amplitudes, and time limits.
- **Buttons:** For triggering convolution/correlation computation, toggling views, and resetting the app.
- **Checkbox and Slider:** To control and adjust the animation behavior and speed.

Axes (UIAxes1--UIAxes4):

- **UIAxes1:** Displays the input signal.
- **UIAxes2:** Shows the impulse response or its shifting during animation.
- **UIAxes3:** Displays the convolution result or correlation depending on toggle.
- **UIAxes4:** Displays the alternate plot when toggled (correlation or convolution).

The animation feature visually demonstrates the convolution process by shifting and flipping the impulse response over the input signal step-by-step.

6 Tool Used

The development environment and tools used in this project include:

- **MATLAB App Designer:**
 - For GUI creation and component management.
 - Provides an integrated environment for drag-and-drop UI layout and backend code.
- **MATLAB Built-in Functions:**
 - `conv()` for convolution.
 - `xcorr()` for cross-correlation.
 - `stem()`, `plot()`, `cla()` for plotting and clearing axes.
 - `timer` object for managing real-time animation.
- **Signal Processing Techniques:**
 - Discrete and continuous-time signal generation.
 - Time-aligned convolution and correlation operations.

7 Team Member Contributions

Member Name	ID	Contribution
Araf Bin Musa	210021227	Convolution algorithm logic & correlation feature coding, report writing and editing, overall debugging.
Mahzabin Ishrat	210021233	Continuous-time signal math logic, GUI signal plotting and report writing
Somia Alam	210021239	GUI design (App Designer), animation logic, and report writing
Balbone Djenabou	210021265	Debugging (UI + logic), test case handling, report writing assistance
LiBAN ABDIRISAQ	210021249	Debugging, report writing assistance

8 Timeline

Phase	Date Range	Description
Team Already Formed	Before 30 May	Team of 5 members was already ready
Project Announcement	30 May	Topic and instructions officially given
Initial Planning & Task Division	31 May – 2 June	Project objectives discussed, individual roles assigned
GUI Design & Signal Input Setup	3 – 10 June	GUI tab design, signal input logic (discrete + continuous)
Convolution Coding Phase	11 – 20 June	Discrete & continuous-time convolution implementation with animation
Correlation Feature & Enhancements	21 – 26 June	Added optional feature for signal correlation and animation speed control
Debugging & Multi-test Validation	27 June – 6 July	UI bug fixes, runtime checks, cross-check output values
Report Writing & Finalization	7 – 13 July	Full documentation, methodology, design and contributions written
Presentation & Submission Prep	14 – 18 July	Final version saved, presentation prepared and submitted